

Mini Project 1

Mutation and State: From Substitution to Store

1. Introduction

In every language we have built so far, variables have behaved in a simple and predictable way. When you write `{let1 {x 5} {+ x x}}`, the variable `x` is bound to the value 5, and that binding never changes. You can think of `x` as a name for the number 5, and wherever `x` appears in the body, you can mentally replace it with 5. This is the substitution model, and it has served us well.

But most programming languages you have used before, such as Python, Java, or C, do not work this way. In those languages, a variable is not just a name for a value. It is a container that holds a value, and that value can be changed over time. When you write `x = 5` followed by `x = 10` in Python, the variable `x` first holds 5, then holds 10. The old value is gone.

This project asks you to add exactly that capability to your language. You will introduce mutation, the ability to change the value of a variable after it has been created. This is a small change in syntax but a profound change in how the interpreter works. The substitution model, where you simply replace variable names with their values, will no longer be sufficient. You will need to build something more powerful: an environment that maps variable names to locations, and a store that maps locations to values.

By the end of this project, you will understand how imperative programming is implemented beneath the surface, and you will see that mutation is not magic. It is a specific, well defined mechanism that requires careful bookkeeping.

2. Required Reading

Before you begin implementation, you must study the following sections from the textbook carefully. Each section builds on the previous one, and skipping ahead will leave you confused about why certain design decisions are necessary.

Pages 133 to 138: Safety and Soundness, Making Memory Explicit.

This section begins with a calculator that has two types of values, numbers and strings. It then makes memory allocation explicit by introducing a vector that represents the computer's memory. Values are no longer Racket datatypes. They become addresses in this memory vector. Numbers are stored by writing them into the next available slot and returning the address. Strings are stored by first writing their length, then writing the character codes one by one. The key observation is that once values are just addresses, the language loses the ability to distinguish what kind of value lives at a given address. You can read any address as a number or as a string, and the system will not complain. This is what makes the language unsafe.

Pages 139 to 141: Recovering Safety.

This section shows how to fix the problem by introducing tags. Every value stored in memory is now preceded by a tag that identifies what kind of value it is. When you try to read a number, the system first checks that the tag at that address is indeed the number tag. If not, it raises an error. This is how real languages work. Every value in Python, Java, or JavaScript carries a tag (or type information) that prevents you from treating a string as a number or vice versa. The section also discusses the cost of safety: checking tags takes time, and storing tags takes space. This motivates the idea of type systems, which can guarantee safety statically and eliminate the need for runtime checks.

Pages 141 to 144: Soundness, Generic Printing, Representation of Numbers.

These pages explain the formal relationship between types and safety. Type soundness means that if the type checker accepts a program, the program will never violate safety at runtime. The section on generic printing shows how tags enable a single function to print any value without knowing its type in advance. The section on number representation explains a clever trick used by real language implementations: small numbers are stored directly inside the address itself using bit patterns, so they never need heap allocation. This is not something you need to implement, but understanding it will help you appreciate why the environment and store model is designed the way it is.

3. Concepts You Will Learn

The environment and store model.

Until now, your interpreter has used either substitution or a simple environment that maps variable names directly to values. With mutation, this model breaks down. Consider what happens when two variables should share the same mutable location. The solution is to split the mapping into two parts. The environment maps variable names to abstract locations (think of them as addresses or indices). The store maps those locations to actual values. When you look up a variable, you first consult the environment to find its location, then consult the store to find the value at that location. When you mutate a variable, you find its location through the environment but you update the store, leaving the environment unchanged.

The difference between binding and assignment.

Binding and assignment look similar on the surface but are fundamentally different operations. Binding, as in `let!`, creates a new variable with a new location in the store. Assignment, as in `set!`, changes the value at an existing location. Binding extends the environment with a new entry. Assignment modifies the store but does not touch the environment. Understanding this distinction is essential. A common mistake is to think that `set!` creates a new variable. It does not. It modifies an existing one.

Sequencing and evaluation order.

In a purely functional language, the order in which you evaluate subexpressions usually does not matter, because there are no side effects. Once you introduce mutation, order matters. The expression `{begin {set! x 10} x}` must evaluate the assignment before it evaluates `x`. If you reversed the order, you would get the old value of `x` instead of the new one. The `begin` construct enforces left to right evaluation and returns the value of the last expression.

Safety and tags.

The textbook sections you will read show what happens when values are stored as raw data without any indication of what they represent. The same sequence of bits can be interpreted as a number, a string, or nonsense. Tags solve this problem by attaching a small label to every value that says what kind of value it is. This is relevant to your project because the store you build will hold tagged values, and every operation that reads from the store must check that the value it gets is the kind of value it expects.

4. What You Must Build

You will extend the language from the desugaring project with two new core constructs and one new sugar construct. The pipeline remains the same: parse, then desugar, then interpret. But the interpreter must change significantly.

New core constructs (handled by the interpreter).

The first new construct is assignment.

```
{set! x e}
```

This evaluates the expression `e`, finds the location of the variable `x` in the environment, and updates the store so that location now holds the new value. The variable `x` must already exist. If it does not, the interpreter should report an error.

The second new construct is sequencing.

```
{begin e1 e2 ... en}
```

This evaluates each expression from left to right and returns the value of the last one. The intermediate values are discarded, but their side effects (such as assignments) persist in the store.

New sugar construct (handled by the desugarer).

You will also add a `while` loop to the surface language.

```
{while cond body}
```

This is syntactic sugar. The desugarer must translate it into constructs the interpreter already knows. The translation is as follows.

```
{while cond body}
```

becomes

```
{if cond
  {begin body {while cond body}}
  0}
```

Notice that the while loop desugars into a recursive structure: if the condition is true, execute the body, then repeat; otherwise, return 0. This means your language does not need a primitive loop mechanism. Loops are just sugar over conditionals and sequencing. The desugarer must apply this transformation recursively until no sugar remains.

5. How the Interpreter Must Change

This is the most important section of the project. The changes to the parser and desugarer are relatively straightforward. The interpreter, however, requires a fundamental redesign.

Before this project.

Your interpreter takes an expression and an environment, and returns a value. The environment maps variable names to values. When you evaluate `{let1 {x 5} body}`, you extend the environment with a new entry mapping `x` to 5, then evaluate the body in that extended environment.

After this project.

Your interpreter takes an expression, an environment, and a store. It returns both a value and a (possibly modified) store. The environment maps variable names to locations. The store maps locations to values. When you evaluate `{let1 {x 5} body}`, you allocate a fresh location in the store, bind that location to the value 5, extend the environment with a mapping from `x` to that location, and then evaluate the body.

When you evaluate `{set! x 10}`, you look up `x` in the environment to find its location, then update the store so that location now maps to 10. The environment does not change. Only the store changes.

This distinction is subtle but essential. The environment is about names. The store is about values. Binding creates a new name. Assignment changes a value.

6. Example Programs and Expected Results

Your implementation must correctly handle all the following programs. For each one, study the expected output and make sure you understand why that is the correct result before you try to implement anything.

Example 1. Simple mutation.

```

{let1 {x 5}
  {begin
    {set! x 10}
    x}}

```

Expected result: 10

The variable `x` is created with the value 5. The assignment changes it to 10. The final expression returns the current value of `x`, which is now 10.

Example 2. Multiple assignments.

```

{let1 {x 1}
  {begin
    {set! x {+ x 1}}
    {set! x {+ x 1}}
    x}}

```

Expected result: 3

The variable starts at 1. The first assignment reads `x` (which is 1), adds 1, and stores 2. The second assignment reads `x` (which is now 2), adds 1, and stores 3. This demonstrates that evaluation order matters.

Example 3. Mutation inside a function.

```

{let1 {x 0}
  {let1 {inc {lam _ {set! x {+ x 1}}}}
    {begin
      {inc 0}
      {inc 0}
      {inc 0}
      x}}}

```

Expected result: 3

The function `inc` captures `x` from its enclosing scope through its closure. Each call to `inc` increments the same `x`. This example shows how closures and mutation interact. The function does not have its own copy of `x`; it shares the same location in the store as the outer code.

Example 4. While loop (sugar).

```

{let1 {x 0}
  {let1 {i 1}
    {begin
      {while {if {+ i {- 0 6}} 1 0}
        {begin
          {set! x {+ x i}}
          {set! i {+ i 1}}}}
      x}}}

```

Expected result: 15

This program computes the sum $1 + 2 + 3 + 4 + 5$ using a while loop. The condition checks whether `i` is not equal to 6 (since we do not have a less than operator, we use the trick of

checking whether $i - 6$ is nonzero). The body adds i to the accumulator x and increments i . Remember that the while loop is not handled by the interpreter. The desugarer translates it into conditionals and recursion before the interpreter ever sees it.

Example 5. Mutation does not affect other bindings.

```
{let1 {x 5}
  {let1 {y x}
    {begin
      {set! x 99}
      y}}}
```

Expected result: 5

When y is bound, the value of x at that moment is 5, so y gets its own location in the store initialized to 5. Later, changing x to 99 does not affect y because y has a different location. This is an important test case because it confirms that `let1` copies the value into a new location rather than creating an alias.

7. Summary of Pipeline Changes

The following table summarizes what changes in each component of the pipeline.

Parser.

You must add new branches to recognize `set!`, `begin`, and `while`. Each one produces a corresponding AST node.

AST.

You must add new constructors for `set!` (taking a variable name and an expression), `begin` (taking a list of expressions), and `while` (taking a condition and a body).

Desugarer.

The desugarer must handle the `while` node by translating it into an `if` expression with a recursive `while` in the then branch. The desugarer must also recurse into all sub-expressions of the new core constructs (`set!` and `begin`).

Interpreter.

This is where the major work happens. You must redesign the interpreter to use an environment plus store model. Every evaluation now returns not just a value but also an updated store. The interpreter must handle `set!` by updating the store and `begin` by threading the store through each expression in sequence.

8. Deliverables

Each team must submit the following.

Language specification.

A document describing the grammar of your extended language, listing all surface forms and all core forms, and explaining which constructs are sugar and which are core.

AST definition.

The complete type definition for your abstract syntax tree, showing all constructors including the new ones.

Desugarer.

The implementation of the desugarer with the while transformation rule.

Interpreter.

The implementation of the interpreter using the environment and store model. You should clearly document how the environment and store are represented and how they interact.

Test cases.

At least five test programs demonstrating mutation, sequencing, the while loop, closures interacting with mutation, and a program where evaluation order affects the result.

Reflection.

A short written discussion (one to two pages) addressing the following questions. What was the hardest part of moving from substitution to the environment and store model? Why does the interpreter now need to return a store alongside a value? What would break if you forgot to thread the store through a begin expression? Why is the while loop not a core construct?